

Sensor code for mic working

```
#include <Arduino.h>
#include <driver/i2s.h>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>
#include <BLEClient.h>

#define SERVICE_UUID          "12345678-1234-1234-1234-1234567890ab"
#define CHARACTERISTIC_UUID   "abcdefab-1234-1234-1234-abcdefabcdef"

// ----- INMP441 wiring -----
#define I2S_WS    D2    // WS
#define I2S_SCK   D3    // SCK
#define I2S_SD    D1    // SD

// ----- I2S config -----
#define I2S_PORT  I2S_NUM_0
#define SAMPLE_RATE 16000
#define BUFFER_SAMPLES 512

static BLEAddress *serverAddress = nullptr;
static bool doConnect = false;
static bool connected = false;
static BLERemoteCharacteristic *pRemoteCharacteristic = nullptr;
static BLEClient *pClient = nullptr;

int32_t samples[BUFFER_SAMPLES];
float smoothRms = 0.0f;
static float exposureIndex = 0.0f;    // moved global so RESET can change it

class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
    void onResult(BLEAdvertisedDevice advertisedDevice) override {
        if (advertisedDevice.haveServiceUUID() &&
            advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))) {
            serverAddress = new BLEAddress(advertisedDevice.getAddress());
            advertisedDevice.getScan()->stop();
        }
    }
};
```

```

        doConnect = true;
    }
}
};

bool connectToServer() {
    if (serverAddress == nullptr) return false;

    pClient = BLEDevice::createClient();

    if (!pClient->connect(*serverAddress)) {
        return false;
    }

    BLERemoteService *pRemoteService = pClient->getService(BLEUUID(SERVICE_UUID));
    if (pRemoteService == nullptr) {
        pClient->disconnect();
        return false;
    }

    pRemoteCharacteristic =
pRemoteService->getCharacteristic(BLEUUID(CHARACTERISTIC_UUID));
    if (pRemoteCharacteristic == nullptr) {
        pClient->disconnect();
        return false;
    }

    connected = true;
    return true;
}

void setupI2S() {
    i2s_config_t i2s_config = {
        .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
        .sample_rate = SAMPLE_RATE,
        .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT,
        .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
        .communication_format = I2S_COMM_FORMAT_STAND_I2S,
        .intr_alloc_flags = 0,
    };

```

```

        .dma_buf_count = 8,
        .dma_buf_len = 256,
        .use_apll = false,
        .tx_desc_auto_clear = false,
        .fixed_mclk = 0
    };

    i2s_pin_config_t pin_config = {
        .bck_io_num = I2S_SCK,
        .ws_io_num = I2S_WS,
        .data_out_num = I2S_PIN_NO_CHANGE,
        .data_in_num = I2S_SD
    };

    i2s_driver_install(I2S_PORT, &i2s_config, 0, NULL);
    i2s_set_pin(I2S_PORT, &pin_config);
    i2s_zero_dma_buffer(I2S_PORT);
}

int computeRMS() {
    size_t bytesRead = 0;
    i2s_read(I2S_PORT, samples, sizeof(samples), &bytesRead, portMAX_DELAY);

    int count = bytesRead / 4;
    if (count <= 0) return 0;

    double sumsq = 0.0;

    for (int i = 0; i < count; i++) {
        int32_t s = samples[i] >> 14;
        sumsq += (double)s * (double)s;
    }

    double mean = sumsq / count;
    int rms = (int)sqrt(mean);
    return rms;
}

void startScan() {

```

```

BLEScan *pBLEScan = BLEDevice::getScan();
pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());
pBLEScan->setActiveScan(true);
pBLEScan->start(0, false);
}

void setup() {
  Serial.begin(115200);
  delay(2000);

  setupI2S();

  BLEDevice::init("");
  startScan();
}

void loop() {
  static unsigned long lastWrite = 0;

  if (doConnect && !connected) {
    if (!connectToServer()) {
      startScan();
    }
    doConnect = false;
  }

  if (connected && millis() - lastWrite > 500) {
    lastWrite = millis();

    // --- NEW: check for RESET command before sending next value ---
    std::string incoming = pRemoteCharacteristic->readValue();
    if (incoming == "RESET") {
      exposureIndex = 0.0f;
      smoothRms = 0.0f;
      Serial.println("RESET command received");
    }

    int rms = computeRMS();
    smoothRms = 0.8f * smoothRms + 0.2f * rms;
  }
}

```

```

    int sendRms = (int)smoothRms;

    float contribution = sendRms / 4000.0f;
    exposureIndex += contribution;
    exposureIndex *= 0.98f;

    if (exposureIndex < 0) exposureIndex = 0;
    if (exposureIndex > 400) exposureIndex = 400;

    int sendValue = (int)exposureIndex;

    String out = String(sendValue);
    pRemoteCharacteristic->writeValue((uint8_t*)out.c_str(), out.length(),
false);

    Serial.print("RMS raw: ");
    Serial.print(rms);
    Serial.print(" | RMS smooth: ");
    Serial.print(sendRms);
    Serial.print(" | sent: ");
    Serial.println(sendValue);
}
}

```

Best Sensor updated

Display working code with bluetooth sensor battery powered WARN orange LED 100+ button reset

```

#include <Arduino.h>
#include <Wire.h>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

```

```
#include <Adafruit_NeoPixel.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C

#define NEOPIXEL_PIN D8
#define NUMPIXELS 1
#define BUTTON_PIN D10

#define SERVICE_UUID          "12345678-1234-1234-1234-1234567890ab"
#define CHARACTERISTIC_UUID "abcdefab-1234-1234-1234-abcdefabcdef"

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
Adafruit_NeoPixel pixel(NUMPIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;
volatile int receivedValue = 0;

bool lastButtonReading = HIGH;
bool buttonState = HIGH;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 40;

bool warnFlashState = false;
unsigned long lastWarnFlashTime = 0;
const unsigned long warnFlashInterval = 300;

void drawScreen(int value) {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);

    bool warnState = (value >= 100);

    display.setTextSize(1);
    display.setCursor(0, 0);
    display.println("Sound Exposure");
```

```
display.setTextSize(1);
display.setCursor(90, 0);
if (deviceConnected) {
    display.println("LINK");
} else {
    display.println("WAIT");
}

if (warnState) {
    display.setTextSize(2);
    display.setCursor(0, 16);
    display.println("WARN");

    display.setTextSize(2);
    display.setCursor(72, 16);
    display.println(value);
} else {
    display.setTextSize(2);
    display.setCursor(0, 18);
    display.print("Val:");
    display.println(value);

    display.setTextSize(1);
    display.setCursor(90, 22);
    if (value < 50) {
        display.println("LOW");
    } else {
        display.println("MED");
    }
}

int barX = 0;
int barY = 50;
int barW = 128;
int barH = 10;

int fillW = map(value, 0, 150, 0, barW);
if (fillW < 0) fillW = 0;
```

```

    if (fillW > barW) fillW = barW;

    display.drawRect(barX, barY, barW, barH, SSD1306_WHITE);
    display.fillRect(barX, barY, fillW, barH, SSD1306_WHITE);

    display.display();
}

class MyServerCallbacks : public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) override {
        deviceConnected = true;
        Serial.println("BLE client connected");
        drawScreen(receivedValue);
    }

    void onDisconnect(BLEServer* pServer) override {
        deviceConnected = false;
        Serial.println("BLE client disconnected");
        BLEDevice::startAdvertising();
        Serial.println("Advertising restarted");
        drawScreen(receivedValue);
    }
};

class MyCharacteristicCallbacks : public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pChar) override {
        std::string value = pChar->getValue();

        if (!value.empty()) {
            receivedValue = atoi(value.c_str());

            Serial.print("Received value: ");
            Serial.println(receivedValue);

            drawScreen(receivedValue);
        }
    }
};

```



```

void setup() {
    Serial.begin(115200);
    delay(2000);

    Wire.begin(D4, D5);
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
        Serial.println("OLED init failed");
        while (true) delay(10);
    }

    pixel.begin();
    pixel.clear();
    pixel.show();

    drawScreen(receivedValue);

    BLEDevice::init("DosimeterDisplay");

    BLEServer *pServer = BLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());

    BLEService *pService = pServer->createService(SERVICE_UUID);

    pCharacteristic = pService->createCharacteristic(
        CHARACTERISTIC_UUID,
        BLECharacteristic::PROPERTY_READ |
        BLECharacteristic::PROPERTY_WRITE
    );

    pCharacteristic->setCallbacks(new MyCharacteristicCallbacks());
    pCharacteristic->setValue("0");

    pService->start();

    BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
    pAdvertising->addServiceUUID(SERVICE_UUID);
    pAdvertising->start();

```

```

    Serial.println("BLE display server ready");
}

void loop() {
    bool reading = digitalRead(BUTTON_PIN);

    if (reading != lastButtonReading) {
        lastDebounceTime = millis();
    }

    if ((millis() - lastDebounceTime) > debounceDelay) {
        if (reading != buttonState) {
            buttonState = reading;

            if (buttonState == LOW) {
                Serial.println("Button pressed -> sending RESET");
                pCharacteristic->setValue("RESET");
                receivedValue = 0;
                warnFlashState = false;
                drawScreen(receivedValue);
            }
        }
    }

    lastButtonReading = reading;

    bool warnState = deviceConnected && (receivedValue >= 100);

    if (warnState) {
        if (millis() - lastWarnFlashTime >= warnFlashInterval) {
            lastWarnFlashTime = millis();
            warnFlashState = !warnFlashState;
            drawScreen(receivedValue);
        }

        if (warnFlashState) {
            pixel.setPixelColor(0, pixel.Color(50, 20, 0)); // orange
        } else {

```

```
        pixel.setPixelColor(0, pixel.Color(0, 0, 0));    // off
    }
} else {
    warnFlashState = false;

    if (!deviceConnected) {
        pixel.setPixelColor(0, pixel.Color(50, 0, 0));    // red
    } else {
        pixel.setPixelColor(0, pixel.Color(0, 50, 0));    // green
    }
}

pixel.show();
delay(20);
}
```